Comparative Strength Assessment of Password-Based Key Derivation Functions

Comprehensive Technical Guide: Problem Context, Technologies Used, and Cryptographic Algorithms (PBKDF2 vs scrypt)

Project Domain:	Cybersecurity / Applied Cryptography	**Algorithms:**	PBKDF2 (RFC 2898) & scrypt (RFC 7914)
Language:	Python 3.11+	**Frontend Framework:**	Streamlit (Interactive UI)
Testing Regime:	3-Trial Empirical Timing Latency	**Visual Analytics:**	Plotly & Pandas Data Engine

1. The Need For This Project (Problem Statement)

In modern computer systems, passwords are the most common form of user authentication. However, storing passwords directly in plain text is catastrophic. Historically, developers attempted to protect passwords by hashing them with standard cryptographic hash functions like **MD5**, **SHA-1**, or **SHA-256**. This approach is fatally flawed due to architectural shifts in modern hardware.

The Threat Landscape:

- **Extreme Hash Speed:** General-purpose hash functions (SHA-256) were designed for digital signatures and data integrity verification, meaning they are engineered to be *as fast as possible*. A single consumer GPU (e.g., RTX 4090) can compute over **10 billion SHA-256 hashes per second** using password-cracking suites like Hashcat.
- **Rainbow Tables & Precomputed Dictionaries:** Without unique salting, attackers precalculate the hashes of billions of common dictionary words and reverse stolen password databases instantaneously.
- **Asymmetric Attacker Advantage:** While a server only needs to verify a password once per login, an offline attacker with leaked hashes can dedicate massive clusters of GPUs, FPGAs, and custom ASICs to test billions of guesses concurrently.

> ***Key Insight:*** *A Password-Based Key Derivation Function (KDF) intentionally introduces computational friction (slowness) and memory complexity to balance the playing field. It transforms low-entropy human passwords into high-entropy cryptographic keys while imposing a heavy resource penalty on brute-force attackers.*

2. Technologies and Tools Used in the Project

This project leverages Python's standard library cryptographic implementations and modern data visualization frameworks to provide an empirical, reproducible testbed:

Technology	Role in Project	Key Functional Contribution
Python 3.11+	Core Programming Language	Provides runtime execution environment, strong typing, and C-extension bindings for high-performance cryptography.

hashlib (Standard Lib)	Cryptographic Engine	Direct access to OpenSSL-backed hashlib.pbkdf2_hmac() and hashlib.scrypt(), avoiding unverified third-party libraries.
Streamlit	Interactive Web UI Framework	Enables dynamic parameter sliders, real-time trial triggering, tabbed layouts, and clean cyber-security dashboard styling.
Plotly & Pandas	Data Analytics & Visualization	Generates multi-run trial bar charts, average latency with error bars, and the Memory vs. Latency Trade-Off frontier plot.
time.perf_counter()	High-Precision Benchmarking	Monotonic clock with sub-microsecond resolution unaffected by system clock adjustments, ensuring accurate execution latency measurements.
os.urandom(16)	Cryptographic Entropy Source	Generates 128 bits of cryptographically strong random salt from OS kernel entropy pool (/dev/urandom or Windows CryptGenRandom).

3. Deep-Dive: Cryptographic Algorithms Explained

The project evaluates two distinct design philosophies in cryptographic key derivation: **CPU-bound iteration** (PBKDF2) versus **Sequential Memory-Hard Functions** (scrypt).

A. PBKDF2 (Password-Based Key Derivation Function 2 - RFC 2898)

PBKDF2 is the classical industry standard formalized in PKCS #5 v2.0 and NIST SP 800-132. It operates by iterating a Pseudorandom Function (PRF), typically **HMAC-SHA256**, over the password and salt for *c* iterations:

- **Internal Operation:** $U_1 = \text{PRF}(\text{Password}, \text{Salt} \parallel \text{block_index})$, and each subsequent round is chained as $U_k = \text{PRF}(\text{Password}, U_{k-1})$. The final output block is the XOR summation $T = U_1 \oplus U_2 \oplus \dots \oplus U_c$.
- **Memory Consumption: Negligible (< 1 KiB).** Each step only requires storing the 32-byte hash buffer from the previous step. The CPU performs pure arithmetic hashing.
- **The Critical Vulnerability:** Because PBKDF2 requires virtually zero RAM, an attacker can build custom ASIC chips or utilize GPUs with thousands of compute cores. Each core can independently evaluate a password without being bottlenecked by memory access.

B. scrypt (Sequential Memory-Hard Key Derivation - RFC 7914)

Engineered by Colin Percival in 2009, **scrypt** was created specifically to eliminate the hardware vulnerability of PBKDF2 by demanding significant amounts of fast random-access memory (RAM).

- **Internal Operation (ROMix Algorithm):** scrypt first uses PBKDF2 to derive an initial block array. It then initializes a large array of pseudorandom data in RAM (Phase 1) and repeatedly reads from and modifies pseudo-randomly selected indices based on previous calculations (Phase 2), utilizing the Salsa20/8 stream cipher core.
- **The Memory Hardness Formula:**

  Memory Required (Bytes) = 128 × r × N

  Where **N** is the CPU/memory cost parameter (power of 2, default: 16,384), **r** is the block size factor (default: 8), and **p** is parallelization.

  In our benchmark: $128 \times 8 \times 16,384 = \mathbf{16,777,216\text{ bytes}} \approx \mathbf{16\text{ MiB}}$.

- **Why Memory Hardness Thwarts Attackers:**

1. **GPU Memory Saturation:** An attacker attempting 10,000 parallel password guesses with scrypt would need $10,000 \times 16\text{ MiB} = 160\text{ GiB}$ of high-speed VRAM, causing GPU failure or heavy serialization.

2. **Economic Cost of ASICs:** Fabricating custom silicon with 16 MiB of high-speed embedded SRAM per core vastly increases chip die size and manufacturing cost, destroying the economic return of building cracking hardware.

Algorithm Comparison Matrix

Feature	PBKDF2 (HMAC-SHA256)	scrypt (Salsa20/8 Core)
Resource Bottleneck	CPU Arithmetic Only	CPU + RAM Memory Bandwidth
Memory Footprint	~0 MiB (Negligible)	~16 MiB in standard demo (128·r·N)
Default Parameters	iterations = 100,000	N = 16384, r = 8, p = 1
GPU Resistance	Low (Millions of hashes/sec on GPUs)	High (GPU VRAM quickly exhausted)
ASIC Resistance	Low (Compact chip design)	Very High (Huge silicon die needed for SRAM)
Python API	hashlib.pbkdf2_hmac()	hashlib.scrypt()

4. Step-by-Step Project Workflow

When the application executes on the host computer, it carries out the following operational steps:

Step 1: Input Collection: The user inputs a sample password (e.g., Demo@12345) or tests custom phrases.

Step 2: Cryptographic Salting: The app generates 16 bytes (128 bits) of true random entropy via os.urandom(16), displayed in hexadecimal notation.

Step 3: Triple-Trial Benchmarking: Both PBKDF2 and scrypt independently derive 32-byte (256-bit) keys across 3 distinct runs. High-resolution timestamps (time.perf_counter()) measure latency for Run 1, Run 2, Run 3, Average, Min, and Max.

Step 4: Real-Time Tabulation: An empirical table renders execution times, latency variance, and architectural attributes.

Step 5: Visual Performance Analytics: Grouped bar charts illustrate per-trial consistency, and a scatter plot maps the Computational Latency vs. Memory Frontier.

Step 6: Cryptographic Material Display: The generated AES-256 compatible keys are displayed in hex for inspection.

5. Academic Viva & Defense Questions

Q1: Why do we measure 3 separate runs instead of just 1?
Answer: CPU execution time is subject to operating system scheduling, background daemon spikes, CPU frequency scaling, and cache warm-up. Taking 3 runs provides empirical validity and allows calculation of mean, min, and jitter.

Q2: Can we just increase PBKDF2 iterations to equal scrypt's security?
Answer: No. Increasing PBKDF2 iterations slows down the legitimate authentication server, but because PBKDF2 requires zero memory, an attacker with an array of thousands of GPU or ASIC cores still retains a massive cost-efficiency advantage. Only memory-hardness directly undermines attacker hardware.

Q3: What are current industry standards?
Answer: OWASP recommends memory-hard algorithms like Argon2id and scrypt. For legacy systems constrained to PBKDF2-HMAC-SHA256, OWASP mandates a minimum of 600,000 iterations.